

ST-GraphAD: Spatio-Temporal Graph Neural Network for Network Intrusion Detection on UNSW-NB15

Ayan Bhaumik

Independent Researcher

mrayanbhaumik@gmail.com | connect@ayanbhaumik.in

Abstract

Network intrusion detection systems must identify malicious behavior whose evidence may be distributed across communicating entities and evolve over time. Conventional flow-based methods commonly treat network records as independent samples, while static graph neural networks model relational structure without explicitly encoding temporal evolution. We present ST-GraphAD, a spatio-temporal graph neural network for node-level intrusion detection on UNSW-NB15. Because the commonly distributed CSV release does not expose explicit source and destination IP addresses, flows are converted into dynamic attributed graphs using documented pseudo-node identifiers derived from protocol, service, and role information. A shared three-layer graph convolutional network encodes the spatial structure of each graph snapshot, and a two-layer, four-head temporal self-attention module models dependencies across sequences of five snapshots. Model selection and early stopping use node-level validation AUC-ROC rather than the degenerate graph-level metric. Under the corrected protocol, the official training CSV is split 85/15 into training and validation partitions, preprocessing transformations are fitted on the training partition only, and the official test CSV is held out for final evaluation. Across five random seeds, ST-GraphAD achieves 0.657 ± 0.078 node-level AUC-ROC and 0.858 ± 0.021 F1-score, compared with 0.612 ± 0.030 AUC and 0.836 ± 0.013 F1 for a static GCN baseline. The mean AUC advantage is 0.045 absolute, while the recall advantage is 0.050 absolute. ST-GraphAD exhibits higher run-to-run variance, so the observed improvement is not presented as statistically established. Graph-level evaluation remains degenerate under majority-vote labeling and is therefore treated only as a diagnostic. The paper documents the graph-construction approximation, experimental protocol, computational cost, and remaining limitations.

Index Terms— network intrusion detection; graph neural networks; spatio-temporal modeling; temporal attention; UNSW-NB15; node-level anomaly detection.

1. Introduction

1.1 Background

Network intrusion detection remains a foundational cybersecurity capability. Attack campaigns can unfold through multiple stages, including reconnaissance, exploitation, lateral movement, command-and-control, and exfiltration. These stages may leave related traces in the communication structure of a network as well as in successive traffic observations. A model that evaluates only isolated flow vectors therefore discards potentially informative context.

Graph neural networks provide a natural framework for relational traffic analysis. In a communication graph, nodes represent entities and edges represent interactions. A graph convolutional encoder can aggregate neighborhood information and thereby encode local structural context. Static graph models, however, process snapshots independently and cannot directly represent how a node's neighborhood or behavioral context changes across observation windows.

1.2 Research Gap

The study addresses three practical gaps: (i) reproducible conversion of the UNSW-NB15 CSV representation into graphs without pretending that missing IP fields are available; (ii) controlled comparison of a spatial-only GCN with a spatio-temporal GNN on identical partitions and sequence units; and (iii) an evaluation protocol that uses node-level metrics for model selection rather than relying on a highly imbalanced graph-level majority-vote target.

1.3 Contributions

- A documented flow-to-graph pipeline using explicit pseudo-node construction for the UNSW-NB15 CSV representation.
- ST-GraphAD, a three-layer GCN followed by a two-layer, four-head temporal self-attention encoder over five graph snapshots.
- A corrected 85/15 train/validation split, training-only preprocessing, and official-test holdout.
- Five-seed node-level evaluation with mean $\pm$ standard deviation, using node-level validation AUC for checkpoint selection and early stopping.
- Transparent reporting of graph-level metric degeneracy, computational cost, and the methodological limitations that remain.

2. Related Work

2.1 Network Intrusion Detection

Paradigm	Representative methods	Limitation in the present context
Signature-based	Snort, Suricata, Zeek	Limited coverage of previously unseen patterns
Classical ML	Random Forest, XGBoost, Autoencoders	Primarily flow-centric representations
Deep learning	CNN, LSTM, Transformer	Often lacks explicit interaction topology
Static GNN	GCN, GAT, GraphSAGE	Does not directly model temporal evolution

Table 1. NIDS paradigms relevant to the proposed approach.

2.2 Graph and Temporal GNNs

GraphSAGE established inductive neighborhood aggregation, while GAT introduced attention-based graph message passing. Temporal graph approaches such as TGAT and TGN extend graph representation learning into the temporal domain. ST-GraphAD follows a discrete-time architecture because the available dataset representation is naturally partitioned into graph snapshots, although the resulting temporal ordering is only a proxy for real time.

2.3 UNSW-NB15 Benchmark Context

Model	AUC	F1	Interpretation
Random Forest	0.89	0.85	Prior flow-level result
XGBoost	0.91	0.87	Prior flow-level result
LSTM	0.88	0.84	Prior sequential-flow result
GCN (static)	0.85	0.82	Prior static-graph result
ST-GraphAD (this study)	0.657 ± 0.078	0.858 ± 0.021	Corrected five-seed node-level result

Table 2. Literature/context comparison. Prior values use different tasks and protocols and are not directly comparable to the present node-level evaluation.

3. Dataset and Problem Formulation

3.1 UNSW-NB15

The UNSW-NB15 benchmark contains network-flow records covering nine attack categories: Generic, Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoors, Shellcode, and Worms. The release used in this study contains 175,341 training flows and 82,332 official test flows.

Partition	Source	Flows	Temporal graphs	Nodes	Edges
Training	Training CSV, 85%	149,039	150	178	298,078
Validation	Training CSV, 15%	26,302	27	174	52,604
Official test	Testing CSV, 100%	82,332	83	172	164,664

Table 3. Dataset and graph statistics for the corrected protocol.

3.2 Preprocessing

The training CSV is partitioned into training and validation subsets using stratification by the binary flow label. Categorical encoders and numerical scaling parameters are fitted on the training partition only and then applied unchanged to validation and official test data. Thus, the official test set is not used for fitting preprocessing transformations, model selection, checkpoint selection, or early stopping.

3.3 Graph Construction

The CSV representation lacks explicit source and destination IP addresses. We therefore construct pseudo-nodes from categorical combinations: a source identifier is formed from protocol and service with a source-role suffix, while a destination identifier is formed from state and service with a destination-role suffix. Each flow contributes an edge between the corresponding pseudo-nodes.

This is a proxy graph rather than a reconstruction of the real network topology. Different real hosts can collapse into the same pseudo-node when their available categorical attributes coincide. The limitation is intrinsic to the CSV representation used in this study and is explicitly retained in the interpretation of the results.

3.4 Node Features and Labels

Feature group	Aggregation	Dimension
Source-side statistics	Mean bytes, packets, load, duration, TTL and loss features	11
Destination-side statistics	Corresponding destination-side means	11
Source categorical modes	Protocol, state, service	3
Destination categorical modes	Protocol, state, service	3
Total	Node feature vector	28

Table 4. Node-feature composition.

A node is labeled as an attack node when at least one incident source-side or destination-side flow is labeled as an attack. This simple rule is reproducible, but it may over-label a node when only one incident flow is malicious.

3.5 Temporal Graph Construction

Flows are ordered according to their row order in the CSV because a reliable event timestamp is not available in the working representation. Consecutive blocks of 1,000 flows form graph snapshots, and five consecutive snapshots form the input sequence for ST-GraphAD. The temporal axis is therefore pseudo-temporal rather than a record of real elapsed time.

4. Proposed ST-GraphAD

4.1 Architecture

ST-GraphAD first applies a shared three-layer GCN to each graph snapshot. The resulting 128-dimensional node embeddings are then arranged across the five-snapshot sequence, padded to a common node dimension, and passed through a two-layer, four-head temporal self-attention module with positional encoding. The final-window temporal embeddings are passed to the node-level classifier; a graph-level head is retained as a secondary diagnostic.

Component	Configuration
GCN spatial encoder	3 layers; hidden dimension 128
Temporal encoder	2 self-attention layers; 4 heads
Temporal sequence	5 graph snapshots
Positional encoding	Sinusoidal
Node classifier	128 → 64 → 2
Graph classifier	Mean pool → 64 → 2
Dropout	0.5

Table 5. ST-GraphAD architecture.

4.2 Spatial Encoder

$$\mathbf{H}^{(l+1)} = \sigma(\mathbf{D}^{-1/2} \mathbf{A} \sim \mathbf{D}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)})$$

where $\mathbf{A} \sim = \mathbf{A} + \mathbf{I}$ adds self-loops, $\mathbf{D} \sim$ is the degree matrix of $\mathbf{A} \sim$, and σ is the ReLU activation. Batch normalization and dropout are used between GCN layers.

4.3 Temporal Attention

For each temporal sequence, node embeddings are padded to the maximum node count and accompanied by a validity mask. Sinusoidal positional encoding supplies temporal order. Each attention block computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

Residual connections and layer normalization are applied after the attention blocks. Only the valid nodes in the final snapshot are passed to the detection head.

4.4 Training Objective

$$\mathbf{L} = \mathbf{L}_{\text{graph}} + \lambda \mathbf{L}_{\text{node}}, \lambda = 0.5$$

The model is optimized with Adam (learning rate 10^{-3} , weight decay 5×10^{-4}). ReduceLROnPlateau and early stopping monitor validation node-level AUC-ROC. Graph-level AUC is never used for model selection.

5. Experimental Setup

5.1 Baseline and Fair Comparison

The GCN-only baseline uses the same spatial encoder and node classification head but omits temporal attention. Both models are trained from the same graph partitions and sequence units so that the principal architectural difference is the temporal modeling component.

5.2 Final Configuration

Parameter	Value
Hidden channels	128
GCN layers	3
Temporal attention layers	2
Attention heads	4
Dropout	0.5
Learning rate	1×10^{-3}
Weight decay	5×10^{-4}
Max epochs	50
Early stopping patience	10
Sequence length	5
Max sequences/epoch	200
Seeds	42, 123, 456, 789, 999
Device	CPU, Apple M4, 16 GB RAM

Table 6. Authoritative Configuration C.

5.3 Evaluation Metrics

The primary evaluation task is node-level binary anomaly detection. We report AUC-ROC, F1, precision, and recall aggregated over the official test sequences. The secondary graph-level metric is reported only to document the failure mode of the majority-vote graph-label formulation.

5.4 Reproducibility Protocol

The final study uses five independent random seeds. The principal table reports mean $\pm$ standard deviation across the five seeds. A single seed-42 result is retained only as a traceability reference; it is not merged with or substituted for the multi-seed aggregate.

6. Results

6.1 Principal Five-Seed Results

Model	AUC-ROC	F1	Precision	Recall
GCN-only	0.612 ± 0.030	0.836 ± 0.013	0.789 ± 0.015	0.889 ± 0.023
ST-GraphAD	0.657 ± 0.078	0.858 ± 0.021	0.791 ± 0.022	0.939 ± 0.037

Table 7. Principal five-seed node-level results on the official test set.

Relative to the GCN baseline, ST-GraphAD improves mean AUC-ROC by 0.045 absolute (4.5 percentage points), mean F1 by 0.022 absolute (2.2 percentage points), mean precision by 0.002 absolute (0.2 percentage points), and mean recall by 0.050 absolute (5.0 percentage points).

The improvement should be interpreted cautiously. ST-GraphAD exhibits greater variance across seeds, with AUC standard deviation 0.078 versus 0.030 for GCN and recall standard deviation 0.037 versus 0.023. No statistical significance test was performed; consequently, the observed mean difference is not presented as a statistically established effect.

6.2 Per-Seed Results

Seed	GCN AUC	GCN F1	ST-GraphAD AUC	ST-GraphAD F1
42	0.5727	0.8424	0.6782	0.8288
123	0.6250	0.8499	0.5581	0.8632
456	0.6232	0.8216	0.5945	0.8453
789	0.5848	0.8186	0.6703	0.8590
999	0.6547	0.8474	0.7832	0.8932

Table 8. Per-seed AUC and F1 values used to compute the principal aggregate.

The per-seed results show that ST-GraphAD does not outperform GCN on every seed. In particular, seed 123 produces lower ST-GraphAD AUC than GCN. This supports the variance caveat and argues against characterizing the method as uniformly superior.

6.3 Training Dynamics

The corrected training curves show decreasing training loss for both models and meaningful node-level validation AUC/F1 trajectories. In the representative seed-42 run, the best validation AUC is approximately 0.979 for GCN and 0.994 for ST-GraphAD, with best checkpoints selected using node-level validation AUC.

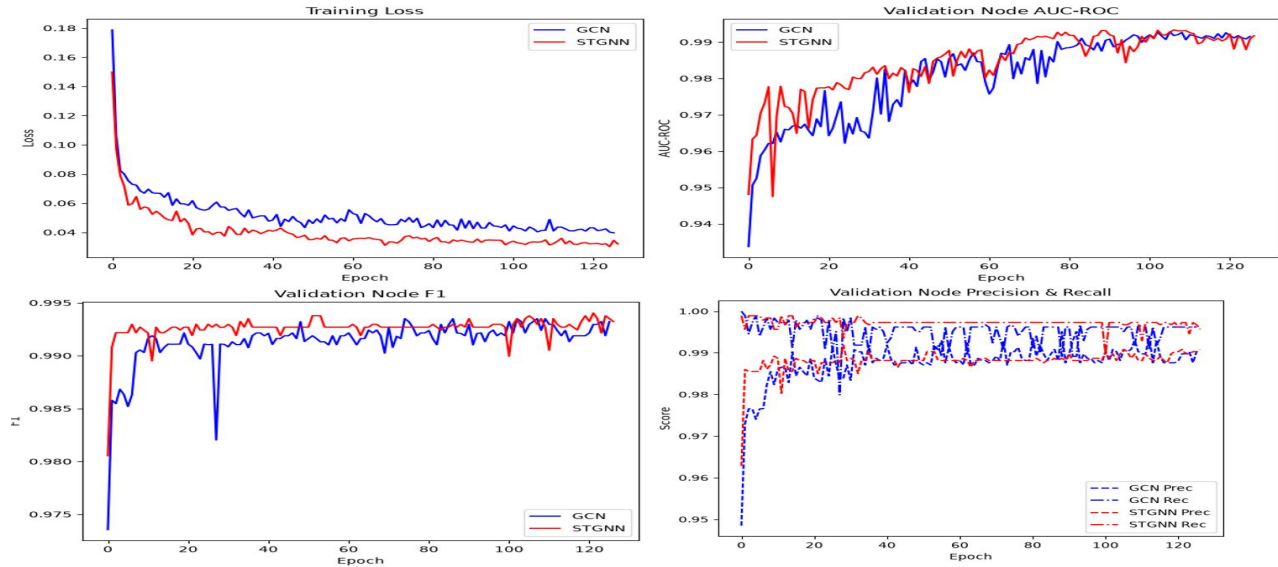


Figure 1. Corrected training loss and node-level validation metrics. The graph-level AUC diagnostic is intentionally excluded from this final figure.

The validation curves should not be interpreted as test-set performance. They are used for checkpoint selection only; the official test set is evaluated after model selection.

6.4 Seed-42 Test Confusion Matrices

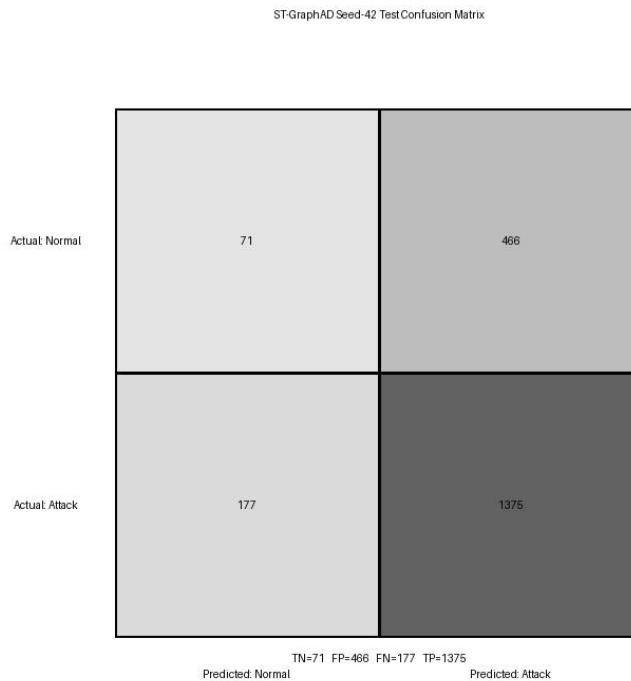


Figure 2. Authoritative ST-GraphAD seed-42 test confusion matrix from Configuration C.

Model	TN	FP	FN	TP
GCN-only, seed 42	99	438	187	1365
ST-GraphAD, seed 42	71	466	177	1375

Table 9. Seed-42 confusion matrices. These are reference diagnostics; confusion matrices vary by seed.

6.5 Graph-Level Evaluation

Model	AUC-ROC	F1	Precision	Recall
GCN-only	0.00	1.00	1.00	1.00
ST-GraphAD	0.00	0.00	0.00	0.00

Table 10. Graph-level metrics under the majority-vote diagnostic formulation.

The graph label is defined by majority vote over the node labels in the final window. With approximately 88% of nodes attack-labeled, almost every graph receives the positive class. The resulting AUC is therefore degenerate, and the apparently perfect F1/precision/recall in the GCN row reflect trivial majority prediction rather than model quality. These metrics are retained only to document the limitation and are not used for checkpoint selection.

6.6 Computational Cost

Model	Parameters	Training time (50 ep., seed 42)	Peak CPU memory
GCN-only	~180K	~10 s	~300 MB
ST-GraphAD	~4.2M	~86 s	~1.2 GB

Table 11. Computational cost for Configuration C.

ST-GraphAD uses roughly 23 times more parameters, approximately four times the peak memory, and approximately 8.6 times the training time of the GCN baseline in the reported 50-epoch seed-42 reference run.

7. Discussion

7.1 Interpretation of the Mean Improvement

The corrected five-seed results are consistent with the hypothesis that temporal context can add useful information to a spatial GCN. The proposed model obtains higher mean AUC, F1, and recall than the baseline. At the same time, the seed-level table demonstrates that the improvement is not uniform across initializations. The larger variance therefore forms an essential part of the result rather than a secondary footnote.

7.2 What the Results Do Not Establish

The study does not establish that particular attention heads detect specific attack stages, nor that the model has empirically demonstrated reconnaissance-to-exploitation tracking or lateral-movement attribution. Those interpretations are design motivations. Quantitative attention analysis and case-level attack tracing were not performed.

7.3 Practical Implications

The node-level formulation is suitable for analyst-oriented prioritization because it produces an anomaly score for individual graph entities. However, the entities are pseudo-nodes rather than real IP identities, and the current computational footprint may be significant for large-scale deployment.

8. Limitations and Threats to Validity

Limitation	Severity	Current status
Pseudo-node construction	High	Unfixed; nodes are categorical proxies, not real hosts.
Pseudo-temporal ordering	High	Unfixed; row order substitutes for actual timestamps.
Fixed 1,000-flow windows	Medium	Unfixed; no domain-based temporal calibration.
Binary node labels	Medium	Only attack vs. normal is evaluated.
Per-attack-category analysis	Medium	Not implemented.
Attention interpretability	Medium	No quantitative head/entropy analysis.
Five-seed variance	Medium	ST-GraphAD AUC std is 0.078 versus 0.030 for GCN.

Single dataset	High	Generalization beyond UNSW-NB15 is untested.
Statistical significance	Medium	No formal hypothesis test or confidence interval is reported.

Table 12. Remaining threats to validity.

Partition independence requires careful interpretation because pseudo-node identifiers can legitimately recur across partitions. Such recurrence is not equivalent to reusing the same flow records. The important leakage controls in the corrected protocol are the separation of flow partitions and fitting of preprocessing transformations on training data only.

9. Reproducibility

The authoritative Configuration C consists of 128 hidden channels, three GCN layers, two temporal attention layers, four heads, sequence length five, dropout 0.5, learning rate 10^{-3} , weight decay 5×10^{-4} , 50 maximum epochs, early stopping patience 10 on node-level validation AUC, and five random seeds: 42, 123, 456, 789, and 999.

The corrected study's principal results are traceable to the per-seed result files and aggregate result file supplied with the project. The exploratory 256-hidden-channel, four-GCN-layer, 200-epoch configuration is treated as a separate experiment and is not used for the headline claims.

10. Conclusion and Future Work

This paper presented ST-GraphAD, a spatio-temporal GNN for node-level network intrusion detection on UNSW-NB15. The corrected protocol uses an 85/15 split of the training CSV, holds the official test CSV out for final evaluation, fits preprocessing on training data only, and uses node-level validation AUC for model selection.

Across five seeds, ST-GraphAD achieves 0.657 ± 0.078 AUC-ROC and 0.858 ± 0.021 F1, compared with 0.612 ± 0.030 AUC and 0.836 ± 0.013 F1 for a static GCN. The corresponding mean improvements are 4.5 percentage points in AUC, 2.2 percentage points in F1, and 5.0 percentage points in recall. The larger variance of ST-GraphAD means the observed advantage should be interpreted as evidence in favor of temporal context, not as a universally established performance gain.

Future work should prioritize real IP/port graph construction from packet captures, event-time or continuous-time temporal modeling, per-attack-category evaluation, formal statistical testing, quantitative attention analysis, and ablation experiments that isolate the contribution of temporal depth, number of heads, sequence length, and hidden size.

References

- [1] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," Military Communications and Information Systems Conference (MilCIS), 2015.
- [2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," International Conference on Learning Representations (ICLR), 2017.
- [3] P. Veličković et al., "Graph Attention Networks," ICLR, 2018.
- [4] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [5] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.
- [6] D. Xu et al., "Inductive Representation Learning on Temporal Graphs," ICLR, 2020.
- [7] E. Rossi et al., "Temporal Graph Networks for Deep Learning on Dynamic Graphs," ICML Workshop, 2020.
- [8] Y. Zhang et al., "Botnet Detection Based on Graph Convolutional Networks," IEEE Access, 2019.
- [9] S. M. Milajerdi et al., "HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows," IEEE Symposium on Security and Privacy, 2019.
- [10] T.-Y. Lin et al., "Focal Loss for Dense Object Detection," ICCV, 2017.

Appendix A. Final Experimental Record

The following material summarizes the final authoritative experiment. It is included so that the manuscript remains traceable to the supplied project records.

Item	Authoritative value
Configuration	C
Seeds	42, 123, 456, 789, 999
Train/validation split	85% / 15% of training CSV
Official test	82,332 flows, untouched until final evaluation
Validation criterion	Node-level AUC-ROC
Headline ST-GraphAD AUC	0.657 ± 0.078
Headline ST-GraphAD F1	0.858 ± 0.021
Headline GCN AUC	0.612 ± 0.030
Headline GCN F1	0.836 ± 0.013

Table A1. Authoritative experimental record.

Appendix B. Graph Construction Pseudocode

```
def construct_graphs(csv, window_size=1000):
    df = read_csv(csv)

    # Fit encoders/scaler on training partition only.
    df = encode_and_scale(df)

    df["src_node"] = df["proto"] + "_" + df["service"] + "_src"
    df["dst_node"] = df["state"] + "_" + df["service"] + "_dst"
    df["window_id"] = df.index // window_size

    graphs = []
    for _, wdf in df.groupby("window_id"):
        if len(wdf) < 10:
            continue
        x = aggregate_node_features(wdf) # 28 dimensions
        y = aggregate_binary_node_labels(wdf)
        edge_index = build_edges(wdf)
        graphs.append(Data(x=x, edge_index=edge_index, y=y))

    return graphs
```